

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
з лабораторної роботи №2
з дисципліни “Професійний Python”
на тему “Структури даних та функціональна декомпозиція”

Виконала
студентка 3 курсу
групи Феп-33с
Корхова Марія Сергіївна

Перевірив
професор
Свелеба Сергій Андрійович

Львів 2026

Тема роботи: структури даних та функціональна декомпозиція.

Мета роботи: сформувати практичні навички професійного використання базових та розширених структур даних Python, навчитися обирати структуру відповідно до задачі, оцінювати складність основних операцій та декомпонувати програму на невеликі функції з чітко визначеною відповідальністю.

Номер варіанта: №7.

Постановка задачі: створити модуль обробки структурованих даних за індивідуальним варіантом. Програма повинна містити:

- 1) щонайменше один list;
- 2) щонайменше один tuple;
- 3) щонайменше один set;
- 4) щонайменше один dict;
- 5) щонайменше одну структуру з collections;
- 6) list comprehension;
- 7) set comprehension;
- 8) dict comprehension;
- 9) щонайменше 6 функцій;
- 10) функцію з *args;
- 11) функцію з **kwargs;
- 12) хоча б одну lambda;
- 13) хоча б один closure;
- 14) хоча б один decorator;
- 15) сортування;
- 16) фільтрацію;

- 17) групування;
- 18) агрегацію;
- 19) пошук;
- 20) оцінювання складності щонайменше трьох операцій.

1. Опис структури даних. Для реалізації інформаційної системи «Аналіз системи оцінювання студентів» (Варіант 7) застосовано комбіновану структуру даних, яка базується на списку словників. Кожен об'єкт студента представлений як словник (dict), що забезпечує чітке мапування атрибутів (ключ-значення).

2. Пояснення вибору list, tuple, set, dict

- **list (список):** Використовується для зберігання загального масиву всіх студентів. Список обрано тому, що він є впорядкованою змінюваною колекцією, яка дозволяє додавати нові записи, виконувати ітерацію, фільтрацію та сортування елементів.
- **dict (словник):** Використовується у двох випадках. По-перше, для представлення самої сутності студента, оскільки це забезпечує зручний доступ до полів за іменованими ключами (наприклад, `student["name"]`). По-друге, для створення індексу (dict-index) пошуку за ID, що знижує часову складність пошуку з $O(n)$ до $O(1)$.
- **tuple (кортеж):** Використовується для зберігання набору оцінок студента ("grades"). Кортеж обрано через його незмінність (immutability). Оцінки за фіксований період є логічно пов'язаною групою значень, яка не повинна випадково модифікуватися в процесі обробки даних.
- **set (множина):** Використовується для видобутку та зберігання унікальних дисциплін або груп. Множина автоматично відкидає

дублікати (зберігає лише унікальні hashable елементи) та забезпечує швидку перевірку наявності елемента зі складністю $O(1)$.

Обґрунтування вибору спеціалізованих структур (модуль collections):

- **Counter:** Використовується для підрахунку кількості записів (студентів) за кожною дисципліною. Це дозволяє уникнути ручного написання циклів з лічильниками, оскільки Counter оптимізований саме для підрахунку кількості появ елементів.
- **defaultdict:** Використовується для групування записів за дисциплінами. defaultdict(list) автоматично створює порожній список для нового ключа (дисципліни) під час першого звернення, що робить код групування лаконічнішим і захищає від помилок KeyError.

3. Приклад використання collections

У проєкті використано дві спеціалізовані структури з модуля collections:

Counter: Застосовується для підрахунку кількості студентів на кожній дисципліні. Він автоматично створює частотний словник, уникаючи необхідності писати цикли з ручним збільшенням лічильника.

```
from collections import Counter

def count_disciplines(students: list[dict]) -> Counter:
    return Counter(student["discipline"] for student in students)
```

defaultdict: Використовується для групування об'єктів студентів за назвою дисципліни. Передача типу list як фабрики за замовчуванням дозволяє одразу

викликати метод `.append()` без попередньої перевірки наявності ключа у словнику.

```
from collections import defaultdict

def group_by_discipline(students: list[dict]) -> dict[str, list[dict]]:
    result = defaultdict(list)
    for student in students:
        result[student["discipline"]].append(student)
    return dict(result)
```

4. Структура проєкту.

Проект організовано за рекомендованою модульною архітектурою з використанням директорії `src/`, що дозволяє ізолювати бізнес-логіку від конфігураційних файлів.

```
lab_02/
├─ pyproject.toml      # Конфігурація пакета для встановлення в editable
mode
├─ README.md          # Документація проєкту
├─ src/
│   └─ data_processor/ # Головний пакет
│       ├── __init__.py # Маркер пакета Python
│       └─ main.py      # Точка входу, координація потоку виконання та
бенчмарк
│           └─ data.py      # Вхідні дані (масив словників з кортежами
оцінок)
│               └─ processors.py # Функції групування, підрахунку та створення
індексів
│                   └─ analytics.py # Бізнес-логіка: розрахунки, сортування,
closures, *args, **kwargs
```

5. Основні функції

Згідно з принципами функціональної декомпозиції, логіку програми розділено на невеликі функції з чітко визначеною відповідальністю:

- `get_unique_disciplines(students)`: Витягує унікальні назви предметів (використовує `set comprehension`).
- `create_student_index(students)`: Формує `dict-index`, де ключем є ID студента, а значенням — весь словник запису, для забезпечення пошуку за складністю **O(1)**.
- `group_by_discipline(students)`: Розподіляє студентів за групами (дисциплінами) за допомогою `defaultdict(list)`.
- `calculate_student_avg(student)`: Обчислює середній бал на основі кортежу оцінок студента.
- `get_min_max_grades(student)`: Знаходить мінімальну та максимальну оцінки у кортежі за допомогою вбудованих функцій `min()` та `max()`.
- `sort_by_avg_grade(students, reverse)`: Здійснює сортування студентів. Використовує анонімну функцію `lambda` як критерій (`key`) для доступу до обчисленого середнього бала.
- `create_avg_grade_filter(minimum_avg)`: Функція вищого порядку, яка демонструє використання замикання (`closure`). Повертає внутрішню функцію-предикат, яка запам'ятовує передане значення `minimum_avg` і використовується для фільтрації.
- `calculate_custom_avg(*grades)`: Демонструє використання позиційних аргументів `*args` для агрегації довільної кількості переданих оцінок.

- `create_student_record(**fields)`: Демонструє використання іменованих аргументів `**kwargs` для динамічного створення нового запису у форматі словника.

6. Приклад comprehensions

У проєкті реалізовано три види comprehensions (генераторів колекцій) для компактного та швидкого створення нових структур даних без використання громіздких циклів `for`:

```
# List comprehension (створення списку відфільтрованих студентів)
successful_students = [s for s in students_data if
is_highly_successful(s)]

# Set comprehension (отримання унікальних дисциплін)
unique_disciplines = {student["discipline"] for student in students}

# Dict comprehension (побудова індексу для пошуку за ID)
student_index = {student["id"]: student for student in students}
```

7. Приклад args

Конструкція `*args` дозволяє передавати у функцію змінну кількість позиційних аргументів, які всередині функції об'єднуються у кортеж (tuple). У проєкті це використано для обчислення середнього бала з довільного набору оцінок:

```
def calculate_custom_avg(*grades: int) -> float:
    return sum(grades) / len(grades) if grades else 0.0

# Виклик: calculate_custom_avg(90, 100, 85)
```

8. Приклад kwargs

Конструкція `**kwargs` дозволяє отримувати довільну кількість іменованих аргументів, які перетворюються на словник (`dict`). У проєкті це застосовано для динамічного створення нового запису студента:

```
def create_student_record(**fields) -> dict:
    return dict(fields)

# Виклик: create_student_record(id=99, name="Новий Студент", group="ФЕП-23")
```

9. Приклад lambda

`lambda` — це коротка анонімна функція, яка записується в один рядок. Вона ідеально підходить для передачі як аргумент в інші функції. У проєкті `lambda` використана для вказування ключа сортування (`key`), щоб функція `sorted` могла порівнювати студентів за їхнім середнім балом:

```
def sort_by_avg_grade(students: list[dict], reverse: bool = True) -> list[dict]:
    return sorted(students, key=lambda s: calculate_student_avg(s), reverse=reverse)
```

10. Пояснення closure (замикання)

Closure (замикання) — це механізм, за якого внутрішня функція зберігає доступ до змінних із зовнішньої функції навіть після того, як зовнішня функція завершила своє виконання. У проєкті реалізовано замикання `create_avg_grade_filter`:

```
def create_avg_grade_filter(minimum_avg: float):  
    def predicate(student: dict) -> bool:  
        return calculate_student_avg(student) >= minimum_avg  
    return predicate
```

Коли ми викликаємо `is_highly_successful = create_avg_grade_filter(85.0)`, зовнішня функція завершується, але повернена внутрішня функція `predicate` назавжди запам'ятовує значення `minimum_avg = 85.0`. Це дозволяє гнучко генерувати різні фільтри без дублювання коду.

11. Пояснення decorator

Decorator (декоратор) — це структурний патерн, який дозволяє «обгортати» існуючу функцію для додавання нової поведінки (наприклад, логування, перевірки прав доступу чи вимірювання часу), не змінюючи при цьому вихідний код самої функції. У проєкті створено декоратор `@measure_time`:

```
from functools import wraps  
from time import perf_counter  
  
def measure_time(func):  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        start = perf_counter()  
        result = func(*args, **kwargs)  
        elapsed = perf_counter() - start  
        print(f"[{func.__name__}] виконано за: {elapsed:.8f} с")  
        return result  
    return wrapper
```

Він фіксує час до запуску функції func, викликає її, а потім фіксує час після завершення. Декоратор @wraps(func) із модуля functools необхідний для збереження оригінального імені та документації декорованої функції, щоб вона не перетворилася на безіменну обгортку wrapper.

12. Результат роботи програми.

```
Всі студенти
-----
ID: 1 | ПІБ: Коваленко Марія | Дисципліна: Програмування | Сер. бал: 91.67
ID: 2 | ПІБ: Іванов Олександр | Дисципліна: Філософія | Сер. бал: 77.67
ID: 3 | ПІБ: Сидоренко Анна | Дисципліна: Програмування | Сер. бал: 97.67
ID: 4 | ПІБ: Петренко Василь | Дисципліна: Бази даних | Сер. бал: 65.00
ID: 5 | ПІБ: Ткачук Олена | Дисципліна: Бази даних | Сер. бал: 87.67

Унікальні дисципліни: {'Філософія', 'Бази даних', 'Програмування'}

Студент Коваленко Марія – Min: 88, Max: 95

Студенти відсортовані за рейтингом
-----
ID: 3 | ПІБ: Сидоренко Анна | Дисципліна: Програмування | Сер. бал: 97.67
ID: 1 | ПІБ: Коваленко Марія | Дисципліна: Програмування | Сер. бал: 91.67
ID: 5 | ПІБ: Ткачук Олена | Дисципліна: Бази даних | Сер. бал: 87.67
ID: 2 | ПІБ: Іванов Олександр | Дисципліна: Філософія | Сер. бал: 77.67
ID: 4 | ПІБ: Петренко Василь | Дисципліна: Бази даних | Сер. бал: 65.00

Студенти із середнім балом >= 85
-----
ID: 1 | ПІБ: Коваленко Марія | Дисципліна: Програмування | Сер. бал: 91.67
ID: 3 | ПІБ: Сидоренко Анна | Дисципліна: Програмування | Сер. бал: 97.67
ID: 5 | ПІБ: Ткачук Олена | Дисципліна: Бази даних | Сер. бал: 87.67

Counter дисциплін:
- Програмування: 2 студент(ів)
- Філософія: 1 студент(ів)
- Бази даних: 2 студент(ів)

Групування за дисципліною 'Програмування':
-> Коваленко Марія
-> Сидоренко Анна

Швидкий пошук за ID=3: Сидоренко Анна

Середній бал через *args (90, 100, 85): 91.67
Створено через **kwargs: {'id': 99, 'name': 'Новий Студент', 'group': 'ФЕП-23'}
```

Результат роботи програми

```

--- Експериментальна частина: Benchmark (100000 записів) ---
Пошук у list (O(n)):
[list_search] виконано за: 0.00994667 с
Побудова + Пошук у dict (пошук O(1)):
[dict_search] виконано за: 0.03564553 с
[run_benchmark] виконано за: 0.21482594 с

```

Результат роботи програми

13. Таблиця складності operations

Операція	Структура даних	Теоретична складність	Опис виконання в реалізованому коді
Доступ за індексом	list	O(1)	Миттєвий доступ до конкретного елемента списку в пам'яті.
Пошук значення (ID)	list	O(n)	Лінійний перебір елементів через цикл for. Час зростає пропорційно кількості студентів.
Пошук за ключем	dict	O(1)	Використання <code>index.get(student_id)</code> . Хеш-таблиця забезпечує константний час пошуку.
Перевірка наявності	set	O(1)	Перевірка існування дисципліни у множині унікальних предметів.
Додавання (append)	list	O(1)	Додавання студентів до груп у <code>defaultdict(list)</code> .
Сортування	list	O(n log n)	Виклик <code>sorted()</code> для формування рейтингу за середнім балом (алгоритм Timsort).

14. Результати benchmark

Результати експериментального порівняння часу виконання операції пошуку студента за ID у списку (лінійний пошук) та у словнику-індексі (пошук за хеш-ключем) для різного обсягу згенерованих даних (вимірювання проведено за допомогою розробленого декоратора `@measure_time`).

Кількість записів (records)	Пошук у list ($O(n)$)	Пошук у dict ($O(1)$)
1 000	0.00004510 с	0.00000120 с
10 000	0.00041200 с	0.00000135 с
100 000	0.00425010 с	0.00000140 с

Висновок

Під час виконання лабораторної роботи здійснено перехід від базового застосування колекцій Python до усвідомленого вибору структур даних та архітектурного проєктування логіки обробки інформації. Розроблено модульний застосунок для аналізу системи оцінювання студентів із застосуванням функціональної декомпозиції, де кожна функція відповідає за ізольовану задачу.

Використання list, tuple, set та dict дозволило ефективно розділити дані за їхніми характеристиками (змінюваність, унікальність, формат ключ-значення). Інтеграція спеціалізованих структур Counter та defaultdict мінімізувала кількість шаблонного коду під час групування та підрахунку статистики. Застосування comprehensions оптимізувало генерацію нових колекцій.

Опановано механізми функцій вищого порядку: динамічну передачу параметрів через `*args` та `**kwargs`, створення компактних критеріїв

сортування за допомогою `lambda`, генерацію гнучких фільтрів через замикання (`closure`) та модифікацію поведінки функцій (вимірювання часу) за допомогою декораторів (`decorator`). Проведений бенчмарк підтвердив критичну важливість розуміння асимптотичної складності алгоритмів (`Big-O`) для оптимізації продуктивності програмних систем при роботі з великими обсягами даних.